

WEEK 2 – PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Scenario:

The e-commerce organization needs to store and manage details about its products and categories. Every product is linked to a particular category, while the system keeps track of important information such as product name, price, and available stock.

1. Design Product and Category Tables

Creating Category Table:

Category table was created with the attributes Category_ID, Category_Name and Description. Category_ID is used to uniquely identify each category.

```
SQL> CREATE TABLE CATEGORY(Category_ID VARCHAR(7),Category_Name VARCHAR(20),Description VARCHAR(50));  
Table created.
```

Creating Product Table:

A Product table was created with the attributes Product_ID, Product_Name, Category_ID, Price and Stock_Quantity. This table stores the details of each product.

```
SQL> CREATE TABLE PRODUCT(Product_ID VARCHAR(7),Product_Name VARCHAR(20),category_ID VARCHAR(7),Price NUMBER(9,2),Stock_quantity NUMBER);  
Table created.
```

2. Define Relationships and Constraints

Defining Primary Key:

Primary keys were added to both tables. Category_ID acts as the primary key in the Category table, while Product_ID acts as the primary key in the Product table. A primary key uniquely identifies each record.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);  
Table altered.
```

Creating Foreign Key Relationship:

The relationship between Category and Product is One-to-Many (1:N). One category can contain many products. The Category_ID in the Product table acts as a foreign key and connects it with the Category table.

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(category_ID);  
Table altered.
```

Adding NOT NULL Constraint:

The NOT NULL constraint was applied to the Category_Name column. It ensures that the category name cannot be left empty.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

Adding CHECK Constraint:

A CHECK constraint was added to ensure that the product price is greater than zero.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (price > 0);  
Table altered.
```

Another CHECK constraint was added to ensure that the stock quantity is not negative.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

Adding UNIQUE Constraint:

A UNIQUE constraint was applied to the Category_Name column. It prevents duplicate category names from being entered into the table.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);  
Table altered.
```

3. Inserting Records

Sample records were inserted into the Category table. The categories include Travel, Sports and Automotive, along with their respective descriptions.

```
SQL> INSERT INTO CATEGORY VALUES (301,'Single','Room for one guest');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (302,'Double','Room for two guests');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (303,'Deluxe','Deluxe room with extra facilities');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (304,'Suite','Spacious premium room');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (305,'Executive','Room with business facilities');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (306,'Family','Room suitable for families');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (307,'Premium','Premium room with modern facilities');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (308,'Economy','Affordable room for guests');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (309,'Twin','Room with two separate beds');
1 row created.

SQL> INSERT INTO CATEGORY VALUES (310,'Presidential','Luxury room with premium facilities');
1 row created.

SQL> SELECT * FROM Category;
```

PRODUCT TABLE:

```
SQL> INSERT INTO PRODUCT VALUES ('P301','Room 101',301,1800,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P302','Room 102',302,2500,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P303','Room 201',303,3500,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P304','Room 202',304,5000,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P305','Room 301',305,3200,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P306','Room 302',306,4000,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P307','Room 401',307,4500,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P308','Room 402',308,1600,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P309','Room 501',309,2800,1);
1 row created.

SQL> INSERT INTO PRODUCT VALUES ('P310','Room 601',310,7500,1);
1 row created.
```

After inserting the records, the contents of both tables can be displayed to verify that the data has been stored successfully.

```
SQL> SELECT * FROM Category;
```

CATEGOR	CATEGORY_NAME	DESCRIPTION
301	Single	Room for one guest
302	Double	Room for two guests
303	Deluxe	Deluxe room with extra facilities
304	Suite	Spacious premium room
305	Executive	Room with business facilities
306	Family	Room suitable for families
307	Premium	Premium room with modern facilities
308	Economy	Affordable room for guests
309	Twin	Room with two separate beds
310	Presidential	Luxury room with premium facilities

```
10 rows selected.
```

```
SQL> SELECT * FROM Product;
```

PRODUCT	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P301	Room 101	301	1800	1
P302	Room 102	302	2500	1
P303	Room 201	303	3500	1
P304	Room 202	304	5000	1
P305	Room 301	305	3200	1
P306	Room 302	306	4000	1
P307	Room 401	307	4500	1
P308	Room 402	308	1600	1
P309	Room 501	309	2800	1
P310	Room 601	310	7500	1

```
10 rows selected.
```

4. CRUD Operations:

CRUD stands for **Create, Read, Update, and Delete**. These are the four fundamental actions used to handle and maintain records in a database. They help users manage product details effectively from adding new information to removing existing records.

Insert Record:

The Insert operation is used to enter a new product into the Product table. It helps in storing the details of products that are newly added or available.

```
SQL> INSERT INTO Product(Product_ID,Product_Name,Category_ID,Price,stock_Quantity) VALUES ('P311','ROOM 61','310',7500,1);
```

```
1 row created.
```

Update Price: The Update operation is used to modify existing product information. In this case, it can be used to change the price of a product when its selling price is revised.

```
SQL> UPDATE Product SET price = 70000 WHERE Product_Name = 'Room 301';  
  
1 row updated.
```

Delete a Product: The Delete operation is used to remove a product entry from the Product table. It is useful when a product is no longer sold, available, or required in the database.

```
SQL> DELETE FROM Product WHERE Product_Name = 'Room 401';  
  
1 row deleted.
```

5. Generate Category-wise Product Reports:

- **Count number of products in each category:** This query is used to find the total number of products available in each category. It helps in understanding how many products are listed under different categories.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
Family	1
Economy	1
Suite	1
Single	1
Deluxe	1
Executive	1
Twin	1
Presidential	2
Double	1

```
9 rows selected.
```

- **Find highest-priced product in each category:** This query is used to identify the most expensive product available in every category. It helps to compare the highest-priced products among the different categories.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price
  2  FROM Product p
  3  JOIN Category c ON p.Category_ID = c.Category_ID
  4  WHERE p.Price = (SELECT MAX(p2.Price)
  5                    FROM Product p2
  6                    WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Single	Room 101	1800
Double	Room 102	2500
Deluxe	Room 201	3500
Suite	Room 202	5000
Executive	Room 301	70000
Family	Room 302	4000
Economy	Room 402	1600
Twin	Room 501	2800
Presidential	Room 601	7500
Presidential	ROOM 61	7500

10 rows selected.

- **Display categories having more than 5 products:** This query is used to show only the categories that contain more than five products. It helps identify categories with a higher number of products.

```
SQL> SELECT c.Category_Name
  2  FROM Product p
  3  JOIN Category c ON p.Category_ID = c.Category_ID
  4  GROUP BY c.Category_Name
  5  HAVING COUNT(p.Product_ID) > 2;
```

no rows selected

- **Find average product price category-wise:** This query is used to calculate the average price of products in each category. It helps understand the typical price of products across different categories.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
Family	4000
Economy	1600
Suite	5000
Single	1800
Deluxe	3500
Executive	70000
Twin	2800
Presidential	7500
Double	2500

9 rows selected.

Conclusion:

The Product and Category Management System offers an organized method for managing e-commerce product details. The use of relationships and constraints helps maintain accurate and consistent data. CRUD operations allow users to add, view, modify, and remove product records. Category-based SQL queries also help in obtaining useful details about products and their pricing.